

HANDS_ON_Flow_Versionierung_Git

Hands-on : Flow-Versionierung mit Git (Flow Registry Client)

Apache NiFi 2.8.0 · UI: <https://localhost:8443/nifi> · Login `admin` `N=/Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0`

Lernziel: Eine Process Group unter **Versionskontrolle** stellen, Änderungen wie Code committen, und auf eine frühere Version **zurückrollen**. NiFi 2.x committet die Flow-Definition (JSON) direkt in ein **Git-Repository** : kein separater NiFi-Registry-Server nötig.

Kerngedanke: Der Flow wird wie Code behandelt: Branches, Pull Requests, Review, Tags, Rollback. Im Repo liegen nur JSON-Dateien.

Voraussetzung: was 2.8.0 mitbringt

In dieser Version sind folgende **Flow Registry Clients** eingebaut (verifiziert in der Installation):

Client	Für
<code>GitHubFlowRegistryClient</code>	GitHub / GitHub Enterprise
<code>GitLabFlowRegistryClient</code>	GitLab (auch self-hosted)
<code>BitbucketFlowRegistryClient</code>	Bitbucket
<code>AzureDevOpsFlowRegistryClient</code>	Azure DevOps

Dazu der klassische NiFi-Registry-Client (gegen einen NiFi-Registry-Server). Wir nehmen hier **GitHub** als Beispiel.

Du brauchst: - ein (leeres) Repository, z.B. `dein-org/nifi-flows`, Branch `main` - einen **Personal Access Token (PAT)** mit Schreibrecht auf das Repo (GitHub: Settings → Developer settings → Fine-grained token, Repo-Scope `Contents: read/write`)

Schritt 1 : Flow Registry Client anlegen

Das ist ein **globaler** Setting, nicht pro Process Group.

1. Oben rechts **Hamburger** `☰` → **Controller Settings**.
2. Tab **Registry Clients** → **+**.
3. Typ `GitHubFlowRegistryClient` auswählen → **Add**.
4. Den Client **bearbeiten** (Stift) → Properties (Namen verifiziert gegen 2.8.0):

Property	Beispielwert	Hinweis
Repository Owner	dein-org	Owner/Org
Repository Name	nifi-flows	Repo-Name
Default Branch	main	nicht "Repository Branch"
Repository Path	flows	darf nicht mit / beginnen/enden
GitHub API URL	https://api.github.com/	nur bei GitHub Enterprise ändern
Authentication Type	Personal Access Token	erst setzen, dann erscheint das Token-Feld
Personal Access Token	dein PAT (Sensitive)	Alternative: GitHub App (App ID + Private Key)

1. **Apply.** Bei korrektem Token ist der Client gültig (kein △).

Stolperstein: PAT ohne `Contents: write` → Commit scheitert beim Speichern. Branch muss existieren (`main` anlegen, falls Repo komplett leer ist). `Repository Path` ohne führenden/ abschließenden `/` (sonst Validierungsfehler "Path can not start or end with /"). NiFi legt beim ersten Versionieren automatisch einen **Default-Bucket** (Unterordner) an.

Schritt 2 : Process Group unter Versionskontrolle stellen

Nimm eine fertige Gruppe, z.B. **"Uebung 1 - GFF-Replace-Update-Put"**.

1. **Rechtsklick** auf die Process Group → **Version** → **Start version control**.
2. Dialog ausfüllen:
 - **Registry Client** = dein GitHub-Client
 - **Bucket** = das **Repository** (der Git-Client mappt Bucket → Repo)
 - **Flow Name** = `uebung1` (wird Datei/Pfad im Repo)
 - **Flow Description** = kurz, optional
 - **Version Comments** = `Initiale Version` (wird zur Git-Commit-Message)
3. **Save.**

NiFi schreibt jetzt die Flow-Definition als JSON nach `flows/` im Repo und legt den ersten Commit an.

Schritt 3 : Die drei Versions-Zustände lesen

Auf der Process Group erscheint ein **Versions-Icon**. Drei Zustände, die du kennen musst:

Icon	Bedeutung	Aktion
grünes Häkchen ✓	lokal == Git, aktuell	nichts zu tun
grauer Stern / Bleistift	lokale, uncommittete Änderungen	Version → Commit local changes

Icon	Bedeutung	Aktion
roter Pfeil nach oben	im Repo gibt es eine neuere Version	Version → Change version (Pull)

Das Icon sitzt links oben an der Process Group bzw. unten in der Statusleiste, wenn du in der Gruppe drin bist.

Schritt 4 : Änderung committen

1. Ändere etwas im Flow (z.B. Run Schedule von GenerateFlowFile auf `60 sec`).
2. Icon wechselt auf **grauer Stern** (lokale Änderung).
3. Rechtsklick auf die Process Group → **Version** → **Commit local changes**.
4. Kommentar `Schedule auf 60s` → **Save** → neuer Commit im Repo.
5. Icon zurück auf **grünes Häkchen**.

Im GitHub-Repo siehst du jetzt zwei Commits auf `flows/uebung1...json`. Genau wie Code-History.

Schritt 5 : Rollback auf eine frühere Version

1. Rechtsklick auf die Process Group → **Version** → **Change version**.
2. Dialog zeigt die **Versionsliste** mit Kommentaren.
3. Wähle die **vorherige** Version (`Initiale Version`) → **Change**.
4. NiFi setzt den Flow auf diesen Stand zurück (Schedule wieder `30 sec`).

So einfach ist der Rollback: keine manuelle Konfiguration, sondern Auswahl einer Version.

Verifiziertes Limit: Der GitHub-Client holt pro Flow **maximal die letzten 10 Commits** (um API-Calls zu sparen). Im Change-version-Dialog siehst du also nur die **letzten 10 Versionen**. Willst du weiter zurück, machst du das per Git direkt (checkout/revert eines älteren Commits) und ziehst dann in NiFi (roter Pfeil → Change version).

Der Git-Workflow drumherum (Empfehlung)

Weil im Repo nur JSON liegt, läuft alles wie bei Code:

- **Pro Process Group** ein Repository **oder** ein Repo + eigener `Repository Path` je PG.
- **Branches:** Client auf `main` (stable) zeigen lassen; Änderungen auf Feature-Branch, dann **Pull Request + Review** in GitHub, dann Merge nach `main`.
- **Tags/Releases** für freigegebene Stände.
- **Rollback** entweder in NiFi (Change version) oder per Git (revert), dann in NiFi pullen (roter Pfeil → Change version).

Erwartetes Ergebnis (Draft : Git-Anbindung nicht live in dieser Umgebung getestet)

- Im Repo `flows/uebung1...json` mit mindestens zwei Commits (`Initiale Version`, `Schedule` auf `60s`).
- Process Group zeigt grünes Häkchen, wenn lokal == Git.
- Change version stellt den alten Stand wieder her.

Hinweis: Die **Property-Namen des GitHub-Clients** sind gegen die 2.8.0-Installation verifiziert (Repository Owner/Name, Default Branch, Repository Path, Authentication Type, Personal Access Token, GitHub API URL). Der **End-to-End-Git-Push** wurde mangels Repo/Token hier nicht durchgespielt; der "Start version control"-Dialog (Bucket-/Flow-Name-Bezeichnungen) ist daher noch nicht verifiziert.

Häufige Stolpersteine

- **Client**  **invalid** → PAT fehlt/abgelaufen oder kein `Contents: write`; Branch existiert nicht.
- **Start version control fehlt im Menü** → du bist nicht auf der Process Group (Rechtsklick muss auf der Gruppe sein, nicht auf leerer Canvas).
- **Commit scheitert** → Token-Rechte, oder Repository Path zeigt auf nicht existierenden Ordner (Git legt Ordner beim ersten Commit an; bei manchen Setups muss der Branch schon da sein).
- **Roter Pfeil bleibt** → jemand hat im Repo committet; mit **Change version** auf neueste ziehen.
- **Sensitive Property leer nach Reimport** → PATs werden nicht im Flow-JSON gespeichert (richtig so); nach Import des Flows auf einer anderen Instanz Token neu setzen.